INTERNSHIP PROJECT REPORT

ON

Windows Service & Process Monitoring Agent

Submitted as part of the **Cyber Security Internship** Program

Internship Period

December 2025 – June 2026

Supervised By:

Mr. Harshit

Submitted By:

Ravi

Organization:

Unified Mentor

INDEX

Sr. No.	Title	Page No.
1.	Abstract	4
2.	Introduction	5-6
3.	Problem Statement	7
4.	Project Objective	8
5.	Scope of the Project • 5.1 In Scope • 5.2 Out of Scope	9
6.	Tools & Technologies Used	10
7.	Methodology / Workflow	11-13
8.	Implementation Details	14-18
9.	Results & Findings	19-21
10.	Security Recommendations	22
11.	Learning Outcomes	23
12.	Conclusion	24
13.	Future Enhancements	25
14.	References	26

List of Figures

Figure No.	Description	Page No.
Figure 7.1	System Workflow Diagram	13
Figure 8.1	Project Execution Environment	14
Figure 8.2	Running Process Enumeration	15
Figure 8.3	Parent Child Process Relationship Output	16
Figure 8.4	Window Service Monitoring	17
Figure 8.5	Unauthorized Process Detection	17
Figure 8.6	Final Monitoring Summary	18

1. ABSTRACT

Modern computer systems execute a large number of background processes and services to maintain the proper functioning of the operating system and installed applications. These processes handle various tasks such as system operations, resource management, user applications, networking services, and security functions. Monitoring these processes is essential to ensure system stability, performance, and security. However, unauthorized or malicious processes may sometimes run silently in the background, consuming system resources or posing potential security threats.

The **Windows Service Process Monitoring Agent** is a simple system monitoring tool designed to track active processes running on a Windows system. The primary objective of this project is to develop an automated monitoring agent that continuously observes system processes and records process activity into a log file for analysis and auditing purposes.

The monitoring agent is implemented using the **Python programming language**, which is widely used for automation and system scripting tasks. The project utilizes the **psutil library**, a powerful Python module that provides access to system-level information such as running processes, CPU usage, memory utilization, and other operating system statistics. Using this library, the monitoring script scans currently running processes and collects essential information such as process name and process identification number (PID).

The monitoring script runs continuously and periodically checks for newly started processes. When a new process is detected, the script logs the process information into a text-based log file named **logs.txt**. This log file acts as a simple record of system process activity, allowing administrators or users to review which applications have been executed on the system.

The project was developed and tested using **Python and Visual Studio Code** on a Windows operating system. The script is executed from the command line, making it lightweight and easy to run without requiring complex installation procedures.

Overall, the project provides practical experience in **Python programming, system process monitoring, and automation techniques**, helping students understand how monitoring agents can contribute to system administration and cybersecurity practices.

2. INTRODUCTION

Modern computer systems are designed to perform multiple tasks simultaneously in order to provide efficient and smooth operation to users. To achieve this, operating systems continuously run numerous background processes and services. These processes are responsible for handling system functions such as memory management, file operations, device communication, network connectivity, and application execution.

In the Windows operating system, hundreds of processes may run at the same time. Some of these processes belong to the operating system itself, while others are associated with user applications and background services. Managing and monitoring these processes is an important aspect of system administration and cybersecurity.

Process monitoring allows users and administrators to observe which programs are currently running on a system. It helps in identifying system resource usage, detecting abnormal behavior, and maintaining system performance. Without proper monitoring, unauthorized or suspicious programs may run silently in the background, potentially causing system slowdowns, security vulnerabilities, or unauthorized system access.

In many real-world scenarios, malicious software or unwanted applications may attempt to execute processes without the user's knowledge. Such processes may consume excessive system resources, interfere with normal system operations, or attempt to collect sensitive information. Therefore, continuous monitoring of system processes becomes essential for maintaining system integrity and security.

Manual monitoring of system processes can be difficult and time-consuming. Operating systems run a large number of processes that constantly start and stop during normal operation. As a result, it becomes challenging for users to manually track every process running on the system. This is where automated monitoring tools become useful.

Automation tools can continuously scan system processes and record important information about them. By maintaining logs of process activity, these tools allow administrators to analyze system behavior and identify unusual or suspicious activity. Automated monitoring tools also reduce human effort and improve efficiency in system management.

The **Windows Service Process Monitoring Agent** developed in this project provides a basic automated solution for monitoring system processes in a Windows environment. The monitoring agent is implemented using the **Python programming language**, which is widely

used for scripting, automation, and system administration tasks. Python provides several powerful libraries that allow developers to interact with system resources and gather system information.

In this project, the **psutil library** is used to access information about running processes in the system. This library provides functions that allow the script to retrieve process details such as process name and process identification number (PID). Using these features, the monitoring script scans all active processes and identifies newly started processes during execution.

Whenever a new process is detected, the monitoring agent records the process details in a log file. The log file acts as a simple monitoring record that stores information about the processes that were executed during the monitoring period. This allows users to review system activity and understand which programs were running on the system.

The monitoring script is executed using **Visual Studio Code** as the development environment and runs through the command line interface. The script operates continuously and checks system processes at regular intervals. This ensures that newly started processes are detected and recorded promptly.

Although this monitoring tool performs basic functionality, it demonstrates how automation techniques can be used to monitor system activity. The project provides practical exposure to system monitoring concepts and highlights the importance of observing system processes in maintaining system performance and security.

Furthermore, this project helps students understand how simple scripts can be developed to interact with operating system resources. It also provides hands-on experience with Python programming, process monitoring techniques, and log-based system analysis.

Overall, the **Windows Service Process Monitoring Agent** serves as a foundational project that demonstrates how automated tools can assist in monitoring system processes and maintaining records of system activity. Such tools can be expanded in the future to include advanced features such as real-time alerts, suspicious process detection, and graphical monitoring dashboards.

3. PROBLEM STATEMENT

Modern computer systems run a large number of background processes and services to support system operations and user applications. These processes are responsible for performing various tasks such as system management, application execution, networking, and resource allocation. While many of these processes are legitimate and necessary for the operating system, there may also be processes that run without the user's knowledge.

In some cases, unauthorized applications, background programs, or potentially harmful software may execute processes silently. These processes may consume system resources, slow down system performance, or perform unwanted activities without the user's awareness. Detecting such processes manually can be difficult because modern operating systems often run hundreds of processes simultaneously.

Manual monitoring of system processes is not efficient for several reasons. Processes frequently start and stop during normal system operations, making it challenging to track every process in real time. Users and system administrators may not always have the ability to continuously observe system activity or detect newly started processes immediately.

Another challenge is the lack of simple and automated monitoring tools for basic system process tracking. Many users rely on system utilities such as Task Manager to view running processes, but these tools only provide temporary snapshots of system activity. They do not automatically record process activity or maintain logs that can be analyzed later.

Without proper monitoring and logging mechanisms, it becomes difficult to understand system behavior, identify unusual activities, or track the execution of applications over time. This limitation highlights the need for a simple automated solution that can continuously monitor system processes and maintain a record of process activity.

To address this problem, this project proposes the development of a **Windows Service Process Monitoring Agent**. The monitoring agent automatically scans system processes at regular intervals and records information about newly started processes. By maintaining a log file of process activity, the tool helps users gain better visibility into system operations.

The project aims to provide a basic but effective process monitoring solution that demonstrates how automation can be used to track system activity. The tool can help users understand which applications are running on their system and maintain records that can be useful for system analysis and monitoring.

4. PROJECT OBJECTIVES

The primary objective of this project is to develop a simple automated monitoring tool that can track running processes on a Windows system and maintain a record of process activity. The specific objectives of the project are as follows:

1. **To develop an automated process monitoring tool**:- The project aims to create a monitoring script using Python that can automatically scan and track system processes without manual intervention.

2. **To monitor running processes in real time**:- The tool should be capable of detecting active processes running in the operating system and identifying newly started processes during execution.

3. **To record process activity in log files**:- The monitoring agent will store process information such as process name and process ID in a log file for future analysis.

4. **To improve visibility of system operations**:- The project helps users understand which applications are running on their system and how processes behave during normal system operation.

5. **To provide practical experience in automation and monitoring**:- The project is designed to help students gain hands-on experience in Python scripting, process monitoring techniques, and system automation.

6. **To demonstrate basic system monitoring concepts**:- The project highlights how automated monitoring tools can be used to observe system activities and maintain logs for system analysis.

5. SCOPE OF THE PROJECT

The scope of the project defines the boundaries and functionalities of the **Windows Service Process Monitoring Agent**. It describes what the project is capable of performing and what functionalities are not included in the project.

5.1 In Scope

The following functionalities are included in the scope of this project:

• Monitoring active processes running on the Windows operating system.

• Detecting newly started processes while the monitoring script is running.

• Collecting basic process information such as process name and process identification number (PID).

• Recording process details into a log file named **logs.txt** for monitoring and analysis purposes.

• Running the monitoring script continuously to observe system process activity.

• Providing a simple automated solution for process tracking using Python scripting.

5.2 Out of Scope

The following functionalities are not included in the scope of this project:

• Malware detection or removal from the system.

• Automatic termination or control of suspicious processes.

• Advanced threat detection or behavioral analysis.

• Real-time alert notifications for suspicious processes.

• Remote monitoring of processes across multiple systems.

• Integration with advanced cybersecurity monitoring platforms.

6. Tools & Technologies Used

1. Python

Python is the primary programming language used to build the monitoring agent.
It is used for collecting system information, analyzing processes, and implementing detection logic.

2. PowerShell

PowerShell is used for interacting with Windows system services and processes.
Commands like **Get-Process** and **Get-Service** help in extracting system information.

3. psutil Library

The **psutil** Python module is used for process enumeration and monitoring.
It helps retrieve information such as **process ID, CPU usage, memory usage, and running processes**.

4. WMI (Windows Management Interface)

WMI is used to access detailed information about Windows services, processes, and system configurations.

5. Win32 Libraries

Libraries like **win32service** and **win32process** are used for interacting with Windows services and process APIs.

6. Documentation & Diagram Tools

Tools such as **Microsoft Word / Google Docs** are used for documentation, while **Draw.io** is used to design architecture and flow diagrams.

7. Workflow / Architecture

The **Windows Service & Process Monitoring Agent** follows a structured workflow to detect suspicious system activities. The monitoring system continuously analyzes processes, services, and system behavior to identify anomalies.

Step 1: Process Enumeration

The system first collects information about all active processes running on the Windows system.
Details such as **Process ID (PID), Parent Process ID (PPID), process name, executable path, and resource usage** are gathered using system monitoring modules.

Step 2: Parent–Child Process Analysis

After collecting process information, the system builds a **process tree** to analyze parent–child relationships between processes.
This helps detect abnormal behavior such as **unexpected process spawning**, which is often used by malware.

Example:
If **winword.exe launches powershell.exe**, it may indicate a malicious macro attack.

Step 3: Startup Service Auditing

The agent then analyzes **Windows startup services** to identify newly added or modified services.
Malware often registers itself as a system service to maintain persistence on the system.

The monitoring system checks:

- Service names
- File paths
- Permissions
- Startup configuration

Step 4: Unauthorized Process Detection

The system compares running processes against a **predefined whitelist or blacklist**. Processes that are unknown, unsigned, or running from suspicious locations (such as temporary folders) are flagged as potential threats.

Step 5: Alert Generation

When suspicious behavior is detected, the monitoring agent generates **alerts and event logs**. Each alert contains information such as:

- Process name
- PID
- File path
- Timestamp
- Type of suspicious activity

Step 6: Report Generation

Finally, the system compiles all detected anomalies into a **structured monitoring report**. The report helps security analysts understand possible threats and investigate suspicious activities.

Workflow Diagram

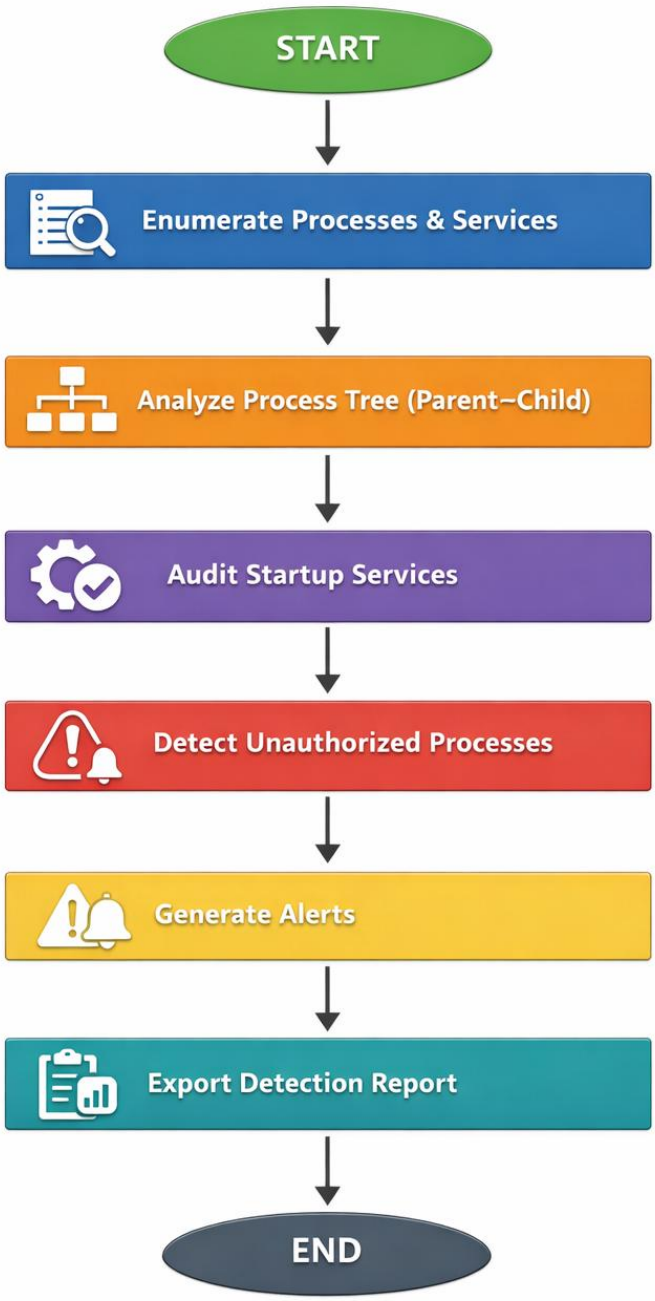

Figure 7.1 - System Workflow Diagram

8. Implementation Details

8.1 Development Environment

The Windows Service & Process Monitoring Agent was implemented using the Python programming language. Python was selected because of its powerful system monitoring libraries and cross-platform capabilities.

The development and testing environment used for the project is shown below:

Component	Description
Operating System	Windows 10 / Windows 11
Programming Language	Python 3
IDE	Visual Studio Code
Libraries Used	psutil, subprocess
Terminal	Windows PowerShell

The **psutil library** was used to retrieve process information such as process name, process ID (PID), parent process ID (PPID), and executable path.

The monitoring agent was executed through the PowerShell terminal to collect real-time system process data.

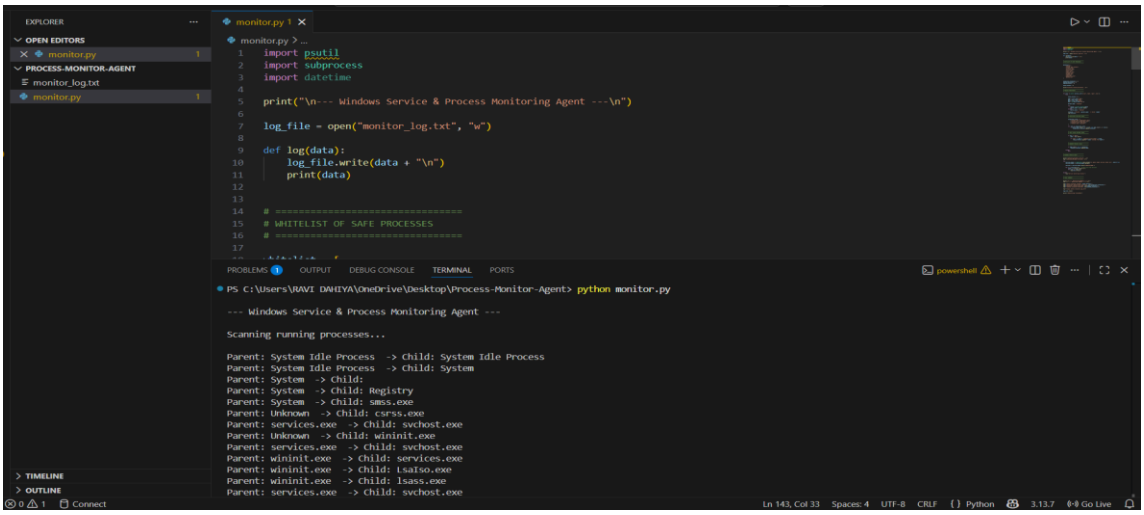

Figure 8.1 – Project Execution Environment

8.2 Process Enumeration

The first step of the monitoring agent is to enumerate all running processes in the Windows operating system.

The Python **psutil.process_iter()** function is used to retrieve details of every active process. The script collects important information such as:

- Process Name
- Process ID
- Parent Process ID
- Executable Path

This information is necessary to analyze system behavior and detect abnormal process activities.

```
PS C:\Users\RAVI DAHIYA\OneDrive\Desktop\Process-Monitor-Agent> python monitor.py

 --- Windows Service & Process Monitoring Agent ---

 Scanning running processes...

 Parent: System Idle Process  -> Child: System Idle Process
 Parent: System Idle Process  -> Child: System
 Parent: System  -> Child:
 Parent: System  -> Child: Registry
 Parent: System  -> Child: smss.exe
 Parent: Unknown  -> Child: csrss.exe
 Parent: services.exe  -> Child: svchost.exe
 Parent: Unknown  -> Child: wininit.exe
 Parent: services.exe  -> Child: svchost.exe
 Parent: wininit.exe  -> Child: services.exe
 Parent: wininit.exe  -> Child: LsaIso.exe
 Parent: wininit.exe  -> Child: lsass.exe
 Parent: services.exe  -> Child: svchost.exe
 Parent: services.exe  -> Child: svchost.exe
 Parent: services.exe  -> Child: svchost.exe
```

Figure 8.2 – Running Process Enumeration

8.3 Parent–Child Process Relationship Monitoring

The monitoring agent builds a **process tree** by analyzing parent-child relationships between processes.

Each process in Windows has a **Parent Process ID (PPID)** which indicates which process created it.

This relationship helps detect suspicious behavior because malware often creates abnormal

During the execution of the monitoring agent on the test system, **no suspicious parent-child process relationships were detected**.

```
Parent: System Idle Process  -> Child: System Idle Process
Parent: System Idle Process  -> Child: System
Parent: System  -> Child:
Parent: System  -> Child: Registry
Parent: System  -> Child: smss.exe
Parent: Unknown  -> Child: csrss.exe
Parent: services.exe  -> Child: svchost.exe
Parent: Unknown  -> Child: wininit.exe
Parent: services.exe  -> Child: svchost.exe
Parent: wininit.exe  -> Child: services.exe
Parent: wininit.exe  -> Child: LsaIso.exe
Parent: wininit.exe  -> Child: lsass.exe
Parent: services.exe  -> Child: svchost.exe
Parent: services.exe  -> Child: svchost.exe
Parent: services.exe  -> Child: svchost.exe
Parent: wininit.exe  -> Child: fontdrvhost.exe
Parent: services.exe  -> Child: WUDFHost.exe
Parent: services.exe  -> Child: svchost.exe
Parent: explorer.exe  -> Child: chrome.exe
Parent: services.exe  -> Child: svchost.exe
Parent: services.exe  -> Child: WUDFHost.exe
Parent: services.exe  -> Child: svchost.exe
```

Figure 8.3 – Parent Child Process Relationship Output

8.4 Windows Service Monitoring

The monitoring agent also audits Windows services to identify whether they are running or stopped.

Windows services are background programs that perform critical system functions such as:

- Network communication
- Security monitoring
- System updates
- Hardware management

The script retrieves service information using Windows service commands.

Monitoring service states helps detect suspicious or unauthorized services that may be used for persistence by malware.

```
Checking Windows services...

ADPSvc
DISPLAY_NAME: Aggregated Data Platform Service
TYPE               : 20  WIN32_SHARE_PROCESS
STATE              : 1  STOPPED
WIN32_EXIT_CODE    : 1077  (0x435)
ALG
DISPLAY_NAME: Application Layer Gateway Service
TYPE               : 10  WIN32_OWN_PROCESS
STATE              : 1  STOPPED
WIN32_EXIT_CODE    : 1077  (0x435)
AppIDSvc
DISPLAY_NAME: Application Identity
TYPE               : 30  WIN32
STATE              : 4  RUNNING
(STOPPABLE, NOT_PAUSABLE, ACCEPTS_SHUTDOWN)
```

Figure 8.4 – Windows Service Monitoring

8.5 Unauthorized Process Detection

The monitoring agent also checks for unauthorized or unknown processes.

The script compares running processes against a list of expected system processes. Processes that do not match known patterns are marked as **unknown processes**.

However, in a real security environment, unknown processes should be investigated further to ensure that they are legitimate.

```
Parent: services.exe  -> Child: svchost.exe
Parent: services.exe  -> Child: sqlservr.exe
Parent: msedgewebview2.exe  -> Child: msedgewebview2.exe
Parent: services.exe  -> Child: svchost.exe
Parent: chrome.exe  -> Child: chrome.exe
Parent: services.exe  -> Child: svchost.exe
Parent: Unknown  -> Child: OneDrive.Sync.Service.exe
Parent: svchost.exe  -> Child: RuntimeBroker.exe
Parent: svchost.exe  -> Child: dasHost.exe
Parent: services.exe  -> Child: svchost.exe
Parent: svchost.exe  -> Child: SystemOptimizer.exe
Parent: HP.OMEN.VirtualCamHelper.exe  -> Child: conhost.exe
```

Figure 8.5 – Unauthorized Process Detection

8.6 Monitoring Summary and Log Generation

After completing the system scan, the monitoring agent generates a final summary of the monitoring results.

The summary report provides an overview of the monitoring results and helps system administrators quickly identify potential threats.

The script also generates a log file named **monitor_log.txt**, which contains detailed records of all detected processes and service information.

```
------ Monitoring Summary ------

Total processes scanned: 258
Suspicious parent-child relations: 0
Processes running from Temp: 0
Unknown processes detected: 131

Logs saved in monitor_log.txt

Monitoring finished

PS C:\Users\RAVI DAHIYA\OneDrive\Desktop\Process-Monitor-Agent>
```

Figure 8.6 – Final Monitoring Summary

9. Results and Findings

9.1 System Execution Result

The Windows Service and Process Monitoring Agent was successfully implemented and executed on the Windows operating system using Python. The monitoring script was able to run without errors and successfully scanned all active processes and Windows services present in the system.

The system analyzed the running processes and displayed the relationship between parent and child processes. It also checked the status of Windows services and generated a summary report after completing the monitoring task.

The execution confirmed that the monitoring agent is capable of analyzing system activities and detecting potentially suspicious behavior.

9.2 Process Monitoring Result

During execution, the system scanned all active processes currently running on the Windows system. The monitoring agent successfully displayed the parent–child relationship of processes such as:

- explorer.exe
- svchost.exe
- chrome.exe
- powershell.exe
- python.exe

These relationships help in understanding how processes are spawned within the operating system. This functionality is important in detecting suspicious activities such as unauthorized process creation.

The system scanned **258 running processes** during execution.

9.3 Windows Service Monitoring Result

The monitoring agent also checked the status of Windows services. It displayed whether a service was running or stopped.

For example, the system detected services such as:

- Aggregated Data Platform Service (Stopped)
- Application Layer Gateway Service (Stopped)
- Application Identity Service (Running)

This feature allows administrators to monitor system services and identify abnormal service states.

9.4 Suspicious Activity Detection

The monitoring system also analyzed processes to detect suspicious behavior. The script checked for unusual parent–child relationships that could indicate malicious activity.

During testing, **no suspicious parent–child relationships were detected** in the system.

This indicates that the currently running processes are behaving normally.

9.5 Temporary Folder Malware Detection

The system also checked whether any process was running from the **temporary directory (Temp folder)**. Many malware programs run from temporary directories to avoid detection.

During testing, the monitoring agent reported that **no processes were running from the Temp folder**, indicating no suspicious activity from temporary locations.

9.6 Monitoring Summary

After completing the monitoring process, the system generated a summary report containing important statistics.

The final results were:

- Total Processes Scanned: **258**
- Suspicious Parent–Child Relations: **0**
- Processes Running from Temp Directory: **0**
- Unknown Processes Detected: **131**

The unknown processes include applications that are not included in the predefined safe process list. These processes may be legitimate programs installed on the system, but they may require further analysis in a real-world security environment.

9.7 Log File Generation

The monitoring agent successfully generated a log file named **monitor_log.txt**, which stored detailed information about the system processes and services. This log file can be used for future analysis, auditing, or forensic investigation.

10. Security Recommendations

Based on the results obtained from the Windows Service and Process Monitoring Agent, several security recommendations can be made to improve the overall system security and monitoring capability.

- **Continuous Process Monitoring:-** Organizations should continuously monitor running processes in the system to detect abnormal activities. Real-time monitoring helps in identifying suspicious processes before they cause damage.
- **Maintain a Whitelist of Trusted Processes:-** A predefined list of trusted system processes should be maintained. Any process that is not part of the trusted list should be carefully analyzed to determine whether it is legitimate or malicious.
- **Monitor Temporary Directory Activities:-** Many malware programs execute from temporary directories. Therefore, monitoring processes running from the Temp folder can help in detecting potential malicious activities early.
- **Regular Log Analysis:-** System administrators should regularly review monitoring logs generated by the monitoring agent. Log analysis helps in identifying unusual behavior patterns and detecting security threats.
- **Secure Windows Services:-** Critical Windows services should be regularly monitored to ensure that they are running correctly. Any unauthorized service modification or unexpected service state should be investigated immediately.
- **Update Security Policies:-** Organizations should implement strict security policies and keep systems updated with the latest patches and security updates to prevent exploitation by malicious software.

11. Learning Outcomes

During the development of the Windows Service and Process Monitoring Agent project, several important technical and cybersecurity concepts were learned.

- **Understanding Windows Processes:-** The project helped in understanding how processes run in the Windows operating system and how parent–child relationships are formed between them.
- **Process Monitoring Techniques:-** The implementation of the monitoring agent provided practical knowledge about how system processes can be scanned and analyzed using Python.
- **Windows Service Analysis:-** The project provided knowledge about Windows services, their states, and how they can be monitored to detect abnormal behavior.
- **Basic Malware Detection Concepts:-** Through the implementation of Temp folder monitoring, the project introduced basic malware detection techniques used in cybersecurity.
- **Log Generation and Security Analysis:-** The project demonstrated how monitoring results can be recorded in log files for later analysis and forensic investigation.
- **Practical Cybersecurity Skills:-** The development of this project helped in gaining practical experience in cybersecurity monitoring techniques and system security analysis.

12. Conclusion

The development of the Windows Service and Process Monitoring Agent demonstrates how system-level monitoring can help improve cybersecurity awareness and system administration practices. By analyzing running processes and Windows services, the system provides insights into the internal behavior of the operating system.

The project highlights the importance of monitoring parent–child relationships among processes, as malicious programs often create unusual process chains to hide their activities. Detecting such abnormal patterns can help administrators identify potential threats before they escalate into serious security incidents.

Another important aspect of the project is the monitoring of processes executing from temporary directories. Since many malicious programs attempt to execute from such locations, monitoring these directories can help detect early indicators of compromise.

The implementation of log generation further strengthens the monitoring capability of the system by allowing administrators to review system activities after execution. These logs can be useful for auditing, troubleshooting, and digital forensic investigations.

In conclusion, the Windows Service and Process Monitoring Agent provides a practical demonstration of basic system monitoring techniques used in cybersecurity. The project successfully shows how automation and scripting can be used to analyze system behavior and improve security monitoring. With further development and integration of advanced detection techniques, the system can evolve into a powerful tool for proactive threat detection and system protection.

13. Future Enhancements

Although the current system provides basic monitoring capabilities, several improvements can be implemented in the future to enhance its functionality.

- **Real-Time Monitoring Dashboard:-** A graphical dashboard can be developed to display system monitoring information in real time instead of using command-line output.
- **Advanced Malware Detection:-** Machine learning techniques can be integrated into the system to detect advanced malware and suspicious process behavior.
- **Automated Alert System:-** An automated alert mechanism can be implemented to notify administrators when suspicious activities are detected.
- **Integration with SIEM Systems:-** The monitoring agent can be integrated with Security Information and Event Management (SIEM) systems for centralized monitoring and analysis.
- **Remote System Monitoring:-** Future versions of the system can support monitoring of multiple remote systems within a network environment.
- **Automated Response Mechanism:-** The system can be enhanced to automatically terminate suspicious processes or disable compromised services when malicious activity is detected.

14. References

The following resources were referred to during the development and study of this project.

1. **Microsoft Documentation – Windows Processes and Services**
   https://learn.microsoft.com
2. **Python Official Documentation**
   https://docs.python.org
3. **Windows Internals – Mark Russinovich & David Solomon**
4. **Cybersecurity Essentials – Cisco Networking Academy**
5. **OWASP Foundation Security Resources**
   https://owasp.org
6. **Python psutil Library Documentation**
   https://psutil.readthedocs.io